

React JS

①

1> What is library? Why we use library?

→ A library is a collection of pre-written code, functions and tools that can be used in the program to make the work easier & faster.

Usage

- * Saves time → You don't have to write the code from scratch. Library gives some ready-made code to do common tasks.
- * Reduces Errors → Library code is usually written by expert developers & is used by 1000's of people, so bugs are found & fixed quickly.
- * Increases Productivity → Build more in less time.

2> What is React JS?

→ React JS is open-source, Javascript frontend library used for building user interfaces quickly, particularly for single-page applications (SPAs)

→ React JS was developed by Jordan Wake in the Year 2011 & open-sourced in March 2013.

→ It is maintained by Meta (formerly known as Facebook) & a Community of developers.

3. List Some features of React JS?

→ i) Component - Based Architecture

ii) React JS uses Virtual DOM

iii) React JS uses JSX (Javascript XML)

iv) Unidirectional Data Flow

v) Declarative UI

vi) Strong Community Support

vii) Reusable Components

viii) Support for Mobile Apps via React Native

4) What is Declarative UI?

→ * We don't need to follow the commands or steps to achieve the output

* It means, you describe what you want to achieve, not how to do it step-by-step.

Feature	React.js	AngularJS
Type	library (for UI)	Full-Fledged Framework
Developed by	Facebook	Google
Language	JavaScript + JSX	JavaScript + HTML ^{or} TypeScript
Architecture	Component-based	MVC [Model-view-Controller]
Data binding	One-way (unidirectional)	Two-way
Learning Curve	Easy to learn & use	Steeper learning Curve
Performance	Faster (due to Virtual DOM)	Slower with large-scale apps
DOM Handling	Virtual DOM	Real DOM

→ 5) Difference between React.js & AngularJS ⁽²⁾

⑥ What is framework? Difference b/w Library & framework?

→ A frame-work is a Pre-defined structure or set of tools that provides a complete environment for Building applications.

→ A framework tells how to build the app, developer should follow the Instructions

Ex Angular, Django, Spring, Express.js

Summary

Library → A library is a Collection of reusable code or functions that help you ^{to} perform specific ^{Tasks} quicker & easier.

* It does not provide a complete structure, and you don't need to follow strict rules.

Framework → A Framework is a Collection of reusable code or functions plus a complete structure and set of rules that you must follow.

∴ Library = Tools only

Framework = Tools + Structure + Rules

⑦ Virtual DOM

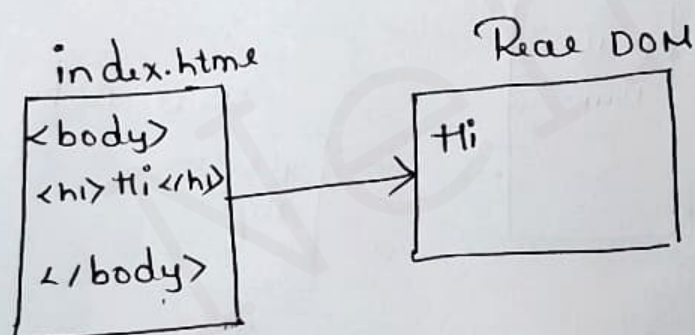
③

In DOM There are 2 Types :

- ① Real DOM (Page will Refresh)
- ② Virtual DOM (Page will not Refresh)

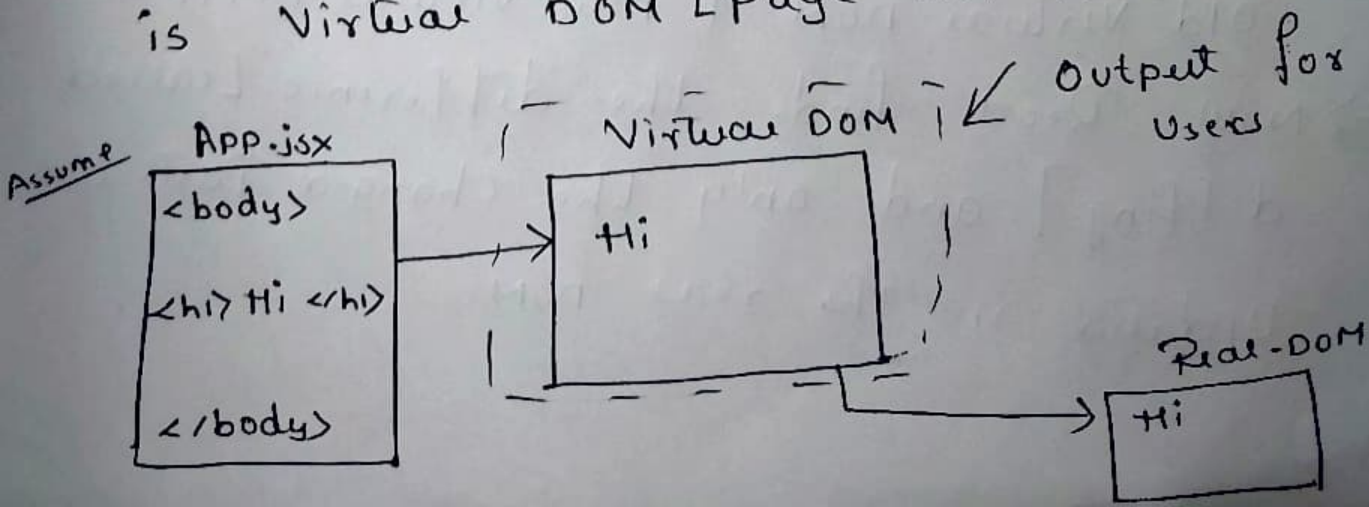
i) Real DOM (Without React.js)

- * Without React.js When we use HTML, CSS, JS, only Real DOM is present.
- * Whatever the user sees in the output is Real DOM [page will refresh]



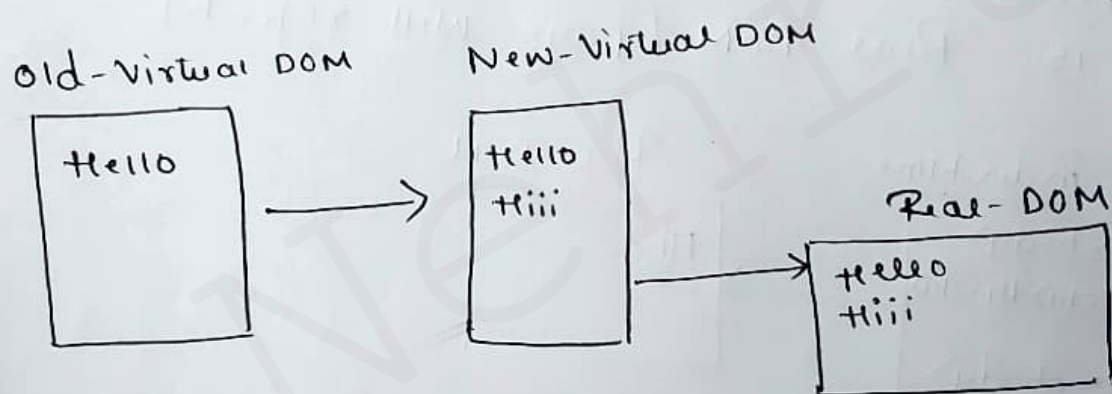
ii) Virtual DOM (With React.js)

- * In React.js both Real DOM as well as Virtual DOM will be present.
- * Whatever the user sees in the output is Virtual DOM [page will not Refresh].



Definition / Explanation

- * Virtual DOM is a Replica of Real-DOM
- * Real DOM & Virtual DOM will be in Sync with the help of Reconciliation & Reconciliation is achieved through diffing Algorithm.
- * Reconciliation Means Comparing old-virtual dom with new-virtual dom



How Reconciliation works?

- Whenever the data is changed in Existing Virtual DOM (i.e. old virtual DOM), React creates a new Virtual DOM and it compares both Old Virtual DOM & new Virtual DOM.
- Now React finds the difference [called 'diffing'] and only the changed parts are updated in the real DOM

⑧ Explain Single Page Application and Multi-Page Application? (4)

→

Multi Page Application (MPA)

→ MPA is a web Application (Frontend + Backend), for each & every request made by the user, the Server loads a new HTML Page.

→ Each time the user clicks a link or buttons, the browser makes a request to the Server, & the Server responds with a Completely new Page

Example : Traditional websites (like old e-commerce sites, university portals etc)
: websites built using PHP, ASP.NET etc.

Disadvantages

- * Slow Navigation
- * Poor user Experience because to load Entire Page it takes some delay
- * Less Mobile friendly because mobile networks

Advantage

- * SEO friendly

Single-Page Application (SPA)

* SPA is a web application (Frontend + backend), that loads a single HTML page once from the server & then dynamically updates the content on the same page without reloading it.

* When you click links or buttons, the page will not reload.

Ex : Gmail, Facebook, Instagram etc..

Advantages

- * Fast Navigation
- * Smooth user Experience, feels like a mobile app
- * Reduces server load, Most logic runs on client side.

Disadvantages

- * Non-SEO friendly [we have to implement manually] [Extra tools are required]

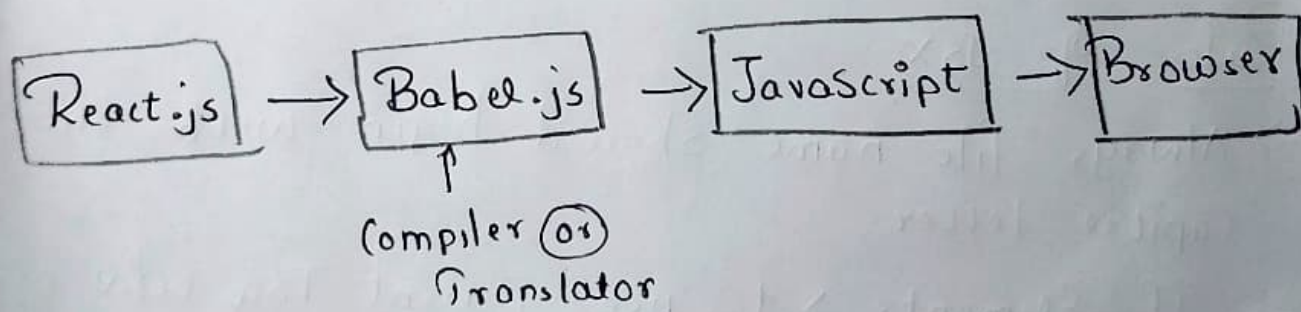
JSX

Q) What is JSX?

- * JSX is similar to HTML
- * JSX is Template Language for React JS
- * JSX is used to Create Components
- * JSX is not understandable by the browser

Why?

- ~~Became~~ Browser understands only Javascript
Because it is traditional language.
- React.js is a Modern Technology,
browser cannot understand.
- Babel.js is like a translator, it will
convert React code into traditional
Javascript & browser can understand.



Defination for JSX

- JSX is a way to write HTML-like code
(not standard HTML) inside Javascript code
to create components.

difference b/w HTML & JSX

HTML	JSX
① <code><!-- Hello --></code>	① <code>{ /* Hello */ }</code>
② <code><h1 class="test">Hi </h1></code>	② <code><h1 className="test">hello </h1></code>
③ <code><h1 id="test">Hi </h1></code>	③ <code><h1 id="test">Hi </h1></code>
④ <code><label for="test">User </label></code>	④ <code><label htmlFor="test"> User </label></code>
⑤ <code><h1 style="color:red"> hello </h1></code>	⑤ <code><h1 style={{color:red}}> hello </h1></code>
⑥ onclick, onchange etc in lowercase	⑥ Use CamelCase like onClick, onChange.

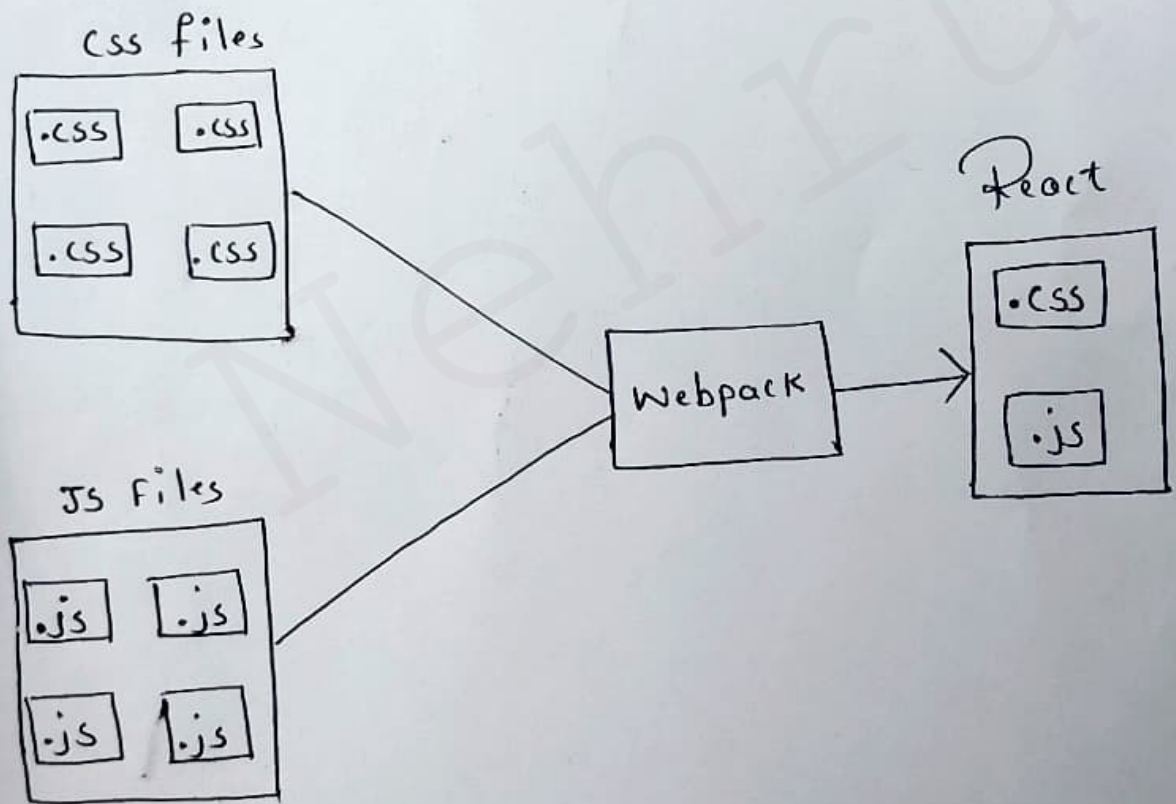
Reels of JSX

- > Always file name should begin with the Capital letter
- > If Elements > 1, use Parent Tag (div @)
- > empty fragments <>
- > Strict Syntax
-> ~~<h1>Hello </h2>~~
~~<h1>Hello </h1>~~
- > JSX Expression is present {}

Webpack

(6)

- Webpack is a powerful Module bundler.
- Module is nothing but a file & bundler means Compressor
- It takes all the files & compresses (or) it bundles them into one file. for optimizing



Alternatives

- 1> Parcel.js
- 2> gulp.js
- 3> browserify

⑪ How To Install React JS?

→ Method 1

Step 1: Install Node.js and npm

1. Go to official Node.js website: <https://nodejs.org>
2. Download the LTS version
3. Install it by running the setup file
4. After installation, open Command Prompt or Terminal & check the versions:

```
> node -v  
> npm -v
```

∴ Note: When you install Node.js, it automatically installs npm with it, so you only need to install Node.js from the official website.

Step 2: Creating a New React App (standard)

① open Command prompt

```
> npx create-react-app app-name
```

↙ must begin with Small letters

∴ wait for few minutes to install all dependencies

```
> cd app-name // move inside the folder.
```

```
> npm start // To start your react app
```


Method 2 (vite Installation)

Step 1 → Install Node.js & npm, follow the Same Step 1 as done in Method 1.

```
> node -v  
> npm -v
```

} verify Installation

Step 2 → Create React app using vite

1. Open Command Prompt or Terminal

```
> npm create vite@latest app-name
```

2. Vite will ask Some questions :

i) Project name → Press Enter (or rename)

ii) Select a framework → Choose : React

iii) Select a variant → choose : JavaScript

3. Go into the Project Folder

```
> cd .
```

```
> cd app-name
```

```
> npm install // install packages
```

4. Run the Development Server

```
> npm run dev
```

⑫ What is Component in React JS,

→ React is a Component Based, meaning it allows us to divide the UI (User Interface) into small pieces of blocks. This block is nothing but a Component which can be reused across the application.

→ Components can be created once & reused multiple times in different parts of an application, reducing code redundancy, saving time & effort in development.

Ex: Write a code for creating Nav-bar only once & call that Nav-bar wherever you want.

Types of Components

There are 2 types of Component

- ① Functional Based Components (Modern Approach)
- ② Class Based Components (Older Approach, now rarely used)

⑬ Difference between Function Based Component and Class Based Component?

→ Function Based	class Based
* It is also known as Stateless, but using hooks we can manage states. * It is simple & Easy * <code>render()</code> method is not required. * <code>Constructor()</code> is not required * <code>'this'</code> keyword is not allowed * Cannot use life cycle methods, but using hooks we can achieve	* It is also known as Stateful Component. * It is Complex, requires more code. * <code>render()</code> method is required. * <code>Constructor()</code> is required. * It needs <code>'this'</code> keyword. * Can use life cycle Method.

Ex Function Based Component

```
import React from "react"
```

```
const Greeting = () => {
```

```
  return (
```

```
    <h1> Hello </h1>
```

```
  )
```

```
}
```

```
export default Greeting;
```

Ex Class Based Component

```
import React from 'react';
```

```
import { Component } from 'react';
```

```
class Nav function name extends Component {
```

```
  render() {
```

```
    return (
```

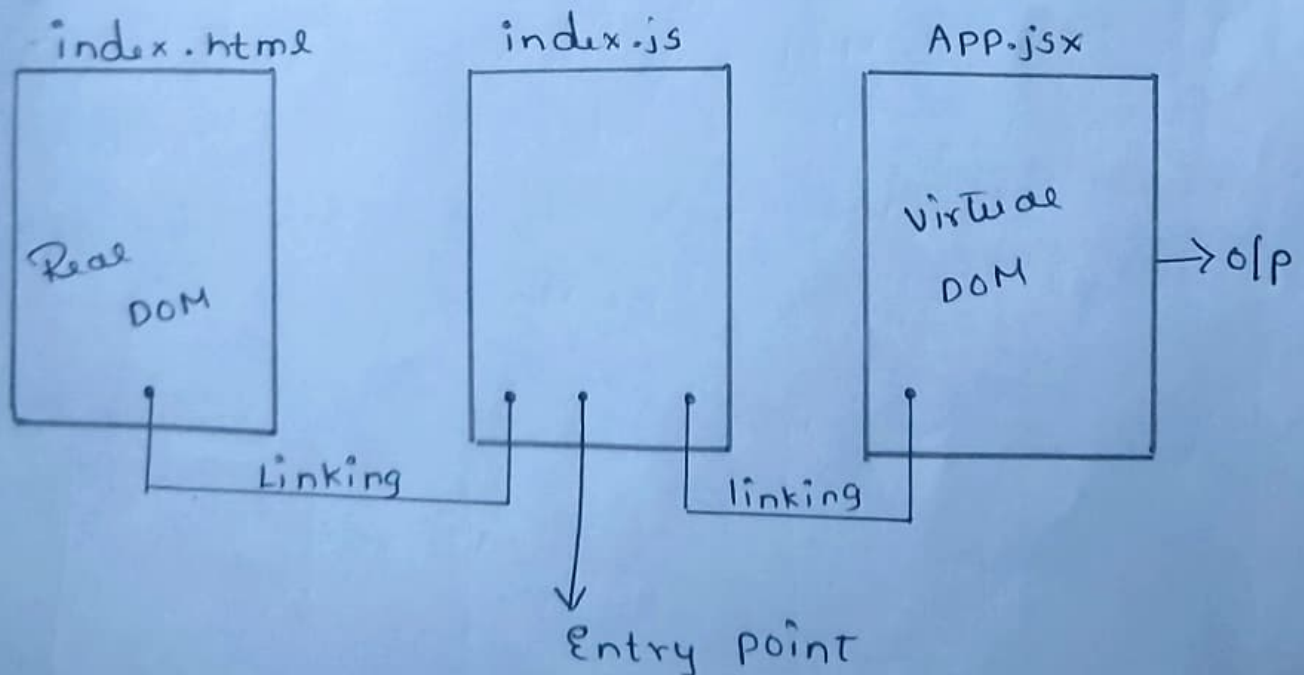
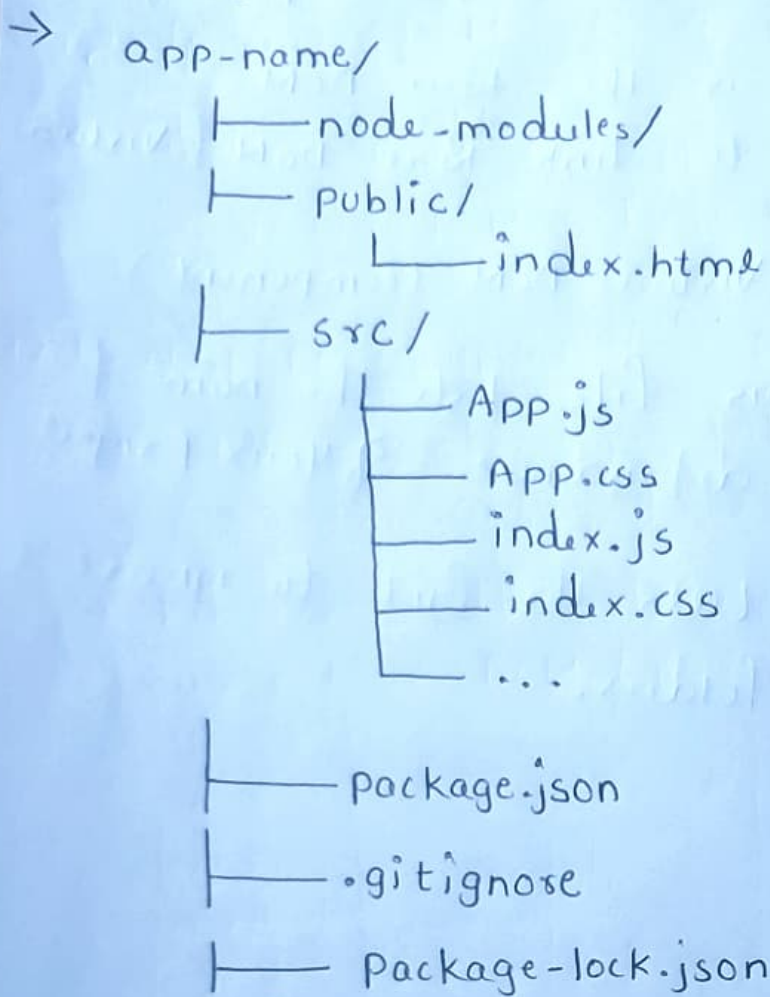
```
      <h1> Hello </h1>
```

```
    )
```

```
}
```

```
export default Nav;
```


13) Project Structure Explanation



- 1) node-modules → Contains all installed packages
- 2) index.html → This is the Real DOM
- 3) index.js → Used to link Real DOM & Virtual DOM
- 4) App.jsx → This is Virtual DOM (component)
- 5) Package.json → This file lists the name & versions of all dependencies of your project.
- 6) .gitignore → it tells the Git to ignore some files or folders.

PROPS

(10)

Props

- Props are nothing but a Properties or attributes.
- Props are ^{way} used to pass data from one Component to another Component. i.e Parent Component to Child Component
- Props are read-only (immutable), the Component which receives data from the another Component, that data cannot be modified.
- Props are uni-direction. i.e data flow in Single-direction.
- Props can be used in both FBC & CBC.

Example:

APP.jsx

```
import React from 'react';
import UserInfo from './UserInfo';

function App() {
  return (
    <UserInfo
      name = "Nehru" age = {333}
    />
  ) </>
};

export default App;
```


UserInfo.jsx

```
import -----  
function UserInfo(props) {  
  return (  
    <>  
      <h2>Name: {props.name} </h2>  
      <p>Age: {props.age} </p>  
    </>  
  )  
}  
export default UserInfo;
```

map(), filter(), reduce()

②

- ① map() → Loops through an array & transforms each Element
∴ Returns a new array
- ② filter() → Filters out Elements based on a Condition
∴ Returns a new array
- ③ reduce() → Performs Calculations on an array
∴ Returns a Single value.

map()

Ex: In JavaScript

```
const numbers = [1, 2, 3, 4, 5]
const doubled = numbers.map(x => {
  return x * 2
});
```

```
console.log(doubled); // [2, 4, 6, 8, 10]
```

Ex: React JS

```
function NameList() {
  const names = ["Nehru", "Raja", "Manoj"]
  return (
```

<>

<h2> Name List : </h2>

{ names.map (cx) => {

return {name}

})

}

</>

);

}

<h2> Name List: </h2>

{ names.map (x) => {

return {name}

})

}

</>

);

}

② filter()

Ex To find numbers > 3

function Details () {

const numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

const filtered = numbers.filter (x) => {

return x > 3

})

return [

<>

{ filtered.map (x) => {

return {x}

})

}

</>

export default Details;

o/p

- 4
- 5
- 6
- 7
- 8
- 9
- 10

③ reduce()

Const Reduce = () => {

const numbers = [10, 20, 30, 40]

const reduced = numbers.reduce((acc, value) => {
 return acc + value
}, 0)

return (
 <>

 {reduced}

</>

)

}

export default Reduce;

o/p : 100

Hooks

(13)

- The Current Version of React is '19.0v'.
- Before the React Version 16.8v, Everyone used Class-Based Component not FBC, Because In CBC there is many features like State Management, Life-Cycle Method etc., but In FBC only few Features were there.
- After the Release of Version 16.8 They Introduced a function 'hooks'.
- Hooks are Special functions in React that allows FBC to use features like state & life cycle Methods.
- Now FBC Can do Everything like a Class Based Component.

Example of Hooks

Hook name	Purpose
useState()	Manages state in function component.
useEffect()	Handles side effects like fetching API, updating DOM
useRef()	Used to Target HTML Elements
useContext()	Provides global state Management (alternative to Props drilling)

Rules of Hooks

* Only call hooks at the top level of a function component

Ex: function {}

function App() {

const [name, setName] = useState("") ✓

if (true) {

const [name, setName] = useState("") ✗

}

}

States

- * State is an object that holds data, allows it to change over time.
- * Whenever the data is changed, this change is captured & stored.
- * States are mutable.

Why Use state?

1. To store & manage dynamic (Ex. user input, API responses)
2. To re-render a component when data changes.
3. To create interactive components (like counters, forms, & toggles)

Syntax

```
import React, {useState} from 'react';  
  
function ComponentName() {  
  const [state, setState] = useState(initialValue)  
  return (  
    <>  
      {state}  
    </>  
  )  
}
```

useRef() Hook

(15)

→ useRef is used to target & manipulate DOM Elements (Read DOM) like `document.getElementById()`, `querySelector()` ... etc.

Use Case	Syntax
To Read input value	<code>current.value</code>
To Change text	<code>current.textContent = "..."</code>
To Change styles	<code>current.style.color = "red"</code>
To Control audio/video	<code>current.play()</code> & <code>current.pause()</code>

Ex : Accessing an `<h1>` Element using `useRef()`

```
import React, {useRef} from 'react';
```

```
const RefExample = () => {
```

```
  const headingRef = useRef(null);
```

```
  const changeColor = () => {
```

```
    headingRef.current.style.color = 'blue';
```

```
    headingRef.current.textContent = "Hi";
```

```
  };
```

```
  return (
```

```
    <>
```

```
    <h1 ref={headingRef}>Hello, </h1>
```

```
    <button onClick={changeColor}>
```

```
      change heading color
```

```
    </button>
```


Forms in React

There are 2 Types of Forms Component

- i> Controlled Form Components
- ii> Uncontrolled Form Components

① Controlled Components

→ A Controlled Component is a form Element whose value is Controlled by React state using 'useState()'

Characteristics

- * React Controls the data entered in the form.
- * Every keystroke updates the state using 'onChange' attribute

② Uncontrolled Components

→ A Uncontrolled Component is a form Element whose value is Controlled by DOM (Real DOM) not by React

→ We use ^{state.} useRef() not useState()

Characteristics

- * DOM handles the data, not React
- * We use 'ref' to read or update the value
- * No onChange (or) state needed.

Ex → Controlled Component

(16)

```
import React, {useState} from 'react'
```

```
const Controlled = () => {
```

```
  const [name, setName] = useState("")
```

```
  function changeName(e) {  
    setName(e.target.value)  
  }
```

```
  return (  
    <>
```

```
    <h1>Name: {name} </h1>
```

```
    <input type="text"
```

```
      value={name}
```

```
      onChange={changeName}
```

```
    /> <br />
```

```
  </>
```

```
)
```

```
}
```

```
export default Controlled;
```

Ex : Uncontrolled Component

```
import React, {useRef} from 'react'

const Uncontrolled = () => {
  const inputRef = useRef(null);

  function changeName() {
    console.log(inputRef.current.value);
  }

  return (
    <>
      <input type="text"
        ref={inputRef} />
      <button onClick={changeName}>
        Submit
      </button>
    </>
  )
}

export default Uncontrolled;
```


Difference between useState & useRef (17)

useState	useRef
* useState deals with Virtual DOM * When the value changes, the component will re-render * The value is stored in React state & displayed on UI * We can display the stored value directly inside JSX * <code>const [name, setName] = useState(" ")</code> * To access value → <code>console.log(name)</code>	* useRef deals with Real DOM * When the value changes, the component will not re-render * Value is stored silently & not reflected on UI. * We cannot display the stored value directly. [We need to use state to display] * <code>const name = useRef(null)</code> * To access value → <code>console.log(name.current.value)</code>

∴ current → object

useEffect()

What is useEffect()?

→ useEffect() is a React hook that allows us to perform side effects in function components.

What is 'Side Effect'?

→ A side effect is anything that happens outside React's Virtual DOM

Examples of side effects

- * Fetching data from an API
- * Interacting with the browser DOM directly.
- * Storing or reading data from local storage.

→ React handles UI, useEffect() handles "external stuff".

→ React will render the component first, later useEffect() will run.

Syntax

`useEffect ( () => { }, [dependencies] ) ;`

Ex : Which will render or runs first.

```
import React, {useEffect} from 'react'
```

```
const UseEffectExample = () => {
```

```
  useEffect ( () => {
```

```
    console.log ("2. UseEffect runs 2nd")
```

```
  } )
```

```
  console.log ("1. Component Renders")
```

```
  return (
```

```
    <h1> Hello </h1>
```

```
  )
```

```
}
```

```
export default UseEffectExample ;
```

o/p

1. Component Renders

2. UseEffect runs 2nd.

Ex: To fetch data from given API

```
import React, { useEffect, useState } from 'react'
```

```
const UseEffectExample = () => {
```

```
  const [users, setUsers] = useState([])
```

```
  useEffect(() => {
```

```
    fetch("https://api.github.com/users")
```

```
    .then(res => res.json())
```

```
    .then(data => setUsers(data))
```

```
  }, [])
```

```
  return (
```

```
    <>
```

```
    <h1> Users: </h1>
```

```
    {for users.map(user => (
```

```
      <div key={user.id}>
```

```
        <img src={user.avatar_url}
```

```
          alt={user.login}
```

```
          width="80"
```

```
          style={{ borderRadius: "50%" }}
```

```
        </div>
```

```
        <p> {user.login} </p>
```

```
      </div>
```

```
    )) }
```

```
    </>
```

```
  )
```

```
} export default UseEffectExample;
```

Routers

- React Router is a library that allows navigation b/w dif Pages in a React Single Page Application.
- Without full page refresh we can navigate b/w dif Pages.
- Routers are used to create dynamic urls in webpage [changes content based on URL]

Installation

> npm install react-router-dom

React Router Components

- ① `<BrowserRouter>` → wraps the Entire app to Enable Routing.
- ② `<Router>` → A Container for all route definitions.
- ③ `<Route>` → Defines a path & the Components to render.
- ④ `<Link>` → Used for navigation without Reloading the Page

Steps to Create Routing

- Step 1: Install > npm install react-router-dom
- Step 2: Enable Routing for the Parent Component
- Step 3: Create paths for the Child Components
- Step 4: Create Link for the Child Components

useNavigate() (Redirecting & Navigation)

- > useNavigate() is a React Router hook used to Programmatically navigate b/w Pages
- > It helps redirect users based on Conditions (ex., after form Submission, authentication, or button clicks)

Ex

```
import {useNavigate} from 'react-router-dom'

function Home() {
  const navigate = useNavigate();
  function goToProfile() {
    navigate("/profile");
  }
  return (
    <>
      <h2> Home Page </h2>
      <button onClick={goToProfile}> Go To
        Profile
      </button>
    </>
  );
}
```

Context API

(22)

Props Drilling

→ Props Drilling is a situation in React where data(props) needs to be passed down multiple levels of components, even if some components don't need that data.

→ To overcome this problem we use Context API

Context API

→ The Context API is a built in feature in React that allows us to share data globally across components without using Props drilling.

Why Use Context API

- * Avoids Props drilling when passing data through multiple levels.
- * Makes data accessible anywhere in the Components
- * Improves code Readability & Maintainability

Steps

- * Import createContext from React
- * Create a Context using createContext()
- * Wrap the Parent/Main Component with the Provider & Pass values
- * Use the value in the Components